Skill

Topic 1 : To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
vennela_07@VennelaPC:~$ mkdir input_redirection
vennela_07@VennelaPC:~$ cd input_redirection
vennela_07@VennelaPC:~/input_redirection$ namo input_redirection.c
Command 'namo' not found, did you mean:
  command 'nano' from snap nano (9.2)
  command 'nano' from deb nano (7.2-2ubuntu0.2)
  command 'nam' from deb nam (1.15-6)
  command 'nemo' from deb nemo (5.8.5-2)
  command 'nama' from deb nama (1.216-2)
  command 'damo' from deb damo (2.1.0-1)
See 'snap info <snapname>' for additional versions.
vennela_07@VennelaPC:~/input_redirection$ nano input_redirection.c
vennela_07@VennelaPC:~/input_redirection$ gcc input_redirection.c -o input_redirection
vennela_07@VennelaPC:~/input_redirection$ echo "Hello from input redirection" > input.txt
vennela_07@VennelaPC:~/input_redirection$ cat input.txt
Hello from input redirection
vennela_07@VennelaPC:~/input_redirection$ ./input_redirection
Enter command with input redirection (<): cat < input.txt

Input redirection successful.
File contents:
Hello from input redirection

Standard input restored.
vennela_07@VennelaPC:~/input_redirection$ ./input_redirection
Enter command with input redirection (<): cat < missing.txt
Error opening input file: No such file or directory
vennela_07@VennelaPC:~/input_redirection$ cat input.txt
Hello from input redirection
```

```
  GNU nano 7.2                                                input_redirection.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char *argv[]) {
    char command[100];
    char filename[100];
    char *symbol;
    int fd;
    int saved_stdin;

    printf("Enter command with input redirection (<): ");
    if (fgets(command, sizeof(command), stdin) == NULL) {
        return 1;
    }

    command[strcspn(command, "\n")] = '\0';

    symbol = strchr(command, '<');

    if (symbol == NULL) {
        printf("Error: Input redirection symbol < not found.\n");
        return 1;
    }

    *symbol = '\0';
```

```
    char *cmd = strtok(command, " \t");
    if (cmd == NULL) {
        printf("Error: Missing command.\n");
        return 1;
    }

    char *args[20];
    int i = 0;
    args[i++] = cmd;

    char *token;
    while ((token = strtok(NULL, " \t")) != NULL && i < 18) {
        args[i++] = token;
    }
    args[i] = NULL;

    char *file_part = symbol + 1;
    while (*file_part == ' ' || *file_part == '\t') {
        file_part++;
    }

    if (*file_part == '\0') {
        printf("Error: Missing input filename.\n");
        return 1;
    }

    if (strchr(file_part, ' ') != NULL ||
        strchr(file_part, '\t') != NULL) {
        printf("Error: Use a single filename without spaces.\n");
        return 1;
    }

    snprintf(filename, sizeof(filename), "%s", file_part);

    fd = open(filename, O_RDONLY);
    if (fd == -1) {
        perror("Error opening input file");
        return 1;
    }


    if (dup2(fd, STDIN_FILENO) == -1) {
        perror("dup2");
        close(fd);
        close(saved_stdin);
        return 1;
    }

    close(fd);

    printf("\nInput redirection successful.\n");
    printf("File contents:\n");

    char buffer[256];
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0) {
        if (write(STDOUT_FILENO, buffer, bytes) == -1) {
            perror("write");
            break;
        }
    }

    if (bytes == -1) {
        perror("read");
    }
```

```c
    if (dup2(saved_stdin, STDIN_FILENO) == -1) {
        perror("Restore stdin");
        close(saved_stdin);
        return 1;
    }

    close(saved_stdin);

    printf("\nStandard input restored.\n");
    return 0;
}
```

Topic 2 : To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```
vennela_07@VennelaPC:~$ mkdir output_redirection
vennela_07@VennelaPC:~$ cd output_redirection
vennela_07@VennelaPC:~/output_redirection$ nano output_redirection.c
vennela_07@VennelaPC:~/output_redirection$ gcc output_redirection.c -o output_redirection
vennela_07@VennelaPC:~/output_redirection$ ./output_redirection
Enter text followed by > filename: Hello Ubuntu > output.txt
Output successfully written to output.txt
vennela_07@VennelaPC:~/output_redirection$ cat output.txt
Hello Ubuntu
vennela_07@VennelaPC:~/output_redirection$ ./output_redirection
Enter text followed by > filename: Welcome to Linux > output.txt
Output successfully written to output.txt
vennela_07@VennelaPC:~/output_redirection$ cat output.txt
Welcome to Linux
vennela_07@VennelaPC:~/output_redirection$ ./output_redirection
Enter text followed by > filename: Hello Ubuntu
Error: Output redirection symbol > not found.
vennela_07@VennelaPC:~/output_redirection$ ./output_redirection
Enter text followed by > filename: Hello Ubuntu > /root/output.txt
Error opening output file: Permission denied
```

GNU nano 7.2 output_redirection.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

int main(void) {
    char input[256];
    char filename[100];
    char *symbol;
    int fd, saved_stdout;

    printf("Enter text followed by > filename: ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    symbol = strchr(input, '>');
    if (symbol == NULL) {
        printf("Error: Output redirection symbol > not found.\n");
        return 1;
    }

    *symbol = '\0';

    char *text = input;
    while (*text == ' ' || *text == '\t')
        text++;

    char *end = text + strlen(text);
    while (end > text && (end[-1] == ' ' || end[-1] == '\t'))
        *--end = '\0';

    char *file = symbol + 1;
    while (*file == ' ' || *file == '\t')
        file++;

    if (*file == '\0') {
        printf("Error: Missing output filename.\n");
        return 1;
    }

    if (strchr(file, ' ') != NULL || strchr(file, '\t') != NULL) {
        printf("Error: Use a filename without spaces.\n");
        return 1;
    }

    snprintf(filename, sizeof(filename), "%s", file);

    saved_stdout = dup(STDOUT_FILENO);
    if (saved_stdout == -1) {
        perror("dup");
        return 1;
    }

    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("Error opening output file");
        close(saved_stdout);
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2");
        close(fd);
```

```c
        close(saved_stdout);
        return 1;
    }

    close(fd);

    if (dprintf(STDOUT_FILENO, "%s\n", text) < 0) {
        perror("Writing output");
        dup2(saved_stdout, STDOUT_FILENO);
        close(saved_stdout);
        return 1;
    }

    if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
        perror("Restore stdout");
        close(saved_stdout);
        return 1;
    }

    close(saved_stdout);

    printf("Output successfully written to %s\n", filename);
    return 0;
}
```

Topic 3 : To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```
vennela_07@VennelaPC:~$ mkdir append_redirection
vennela_07@VennelaPC:~$ cd append_redirection
vennela_07@VennelaPC:~/append_redirection$ nano append_redirection.c
vennela_07@VennelaPC:~/append_redirection$ gcc append_redirection.c -o append_redirection
vennela_07@VennelaPC:~/append_redirection$ ./append_redirection
Enter text followed by >> filename: First line >> output.txt
Text successfully appended to output.txt
vennela_07@VennelaPC:~/append_redirection$ ./append_redirection
Enter text followed by >> filename: Second line >> output.txt
Text successfully appended to output.txt
vennela_07@VennelaPC:~/append_redirection$ cat output.txt
First line
Second line
vennela_07@VennelaPC:~/append_redirection$ ./append_redirection
Enter text followed by >> filename: Hello Ubuntu
Error: Append operator >> not found.
vennela_07@VennelaPC:~/append_redirection$ ./append_redirection
Enter text followed by >> filename: Third line >>
Error: Missing filename.
vennela_07@VennelaPC:~/append_redirection$ ./append_redirection
Enter text followed by >> filename: New file content >> newfile.txt
Text successfully appended to newfile.txt
vennela_07@VennelaPC:~/append_redirection$ cat newfile.txt
New file content
```

GNU nano 7.2 append_redirection.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(void) {
    char input[256];
    char filename[100];
    char *symbol;
    int fd, saved_stdout;

    printf("Enter text followed by >> filename: ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    symbol = strstr(input, ">>");

    if (symbol == NULL) {
        printf("Error: Append operator >> not found.\n");
        return 1;
    }

    *symbol = '\0';

    char *text = input;
    while (*text == ' ' || *text == '\t')
        text++;

    char *end = text + strlen(text);
    while (end > text && (end[-1] == ' ' || end[-1] == '\t'))
        *--end = '\0';

    char *file = symbol + 2;
    while (*file == ' ' || *file == '\t')
        file++;

    if (*text == '\0') {
        printf("Error: Missing text to append.\n");
        return 1;
    }

    if (*file == '\0') {
        printf("Error: Missing filename.\n");
        return 1;
    }

    if (strchr(file, ' ') != NULL || strchr(file, '\t') != NULL) {
        printf("Error: Use a filename without spaces.\n");
        return 1;
    }

    snprintf(filename, sizeof(filename), "%s", file);

    fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }
```

```
        return 1;
    }

    saved_stdout = dup(STDOUT_FILENO);

    if (saved_stdout == -1) {
        perror("dup");
        close(fd);
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2");
        close(fd);
        close(saved_stdout);
        return 1;
    }

    close(fd);

    if (dprintf(STDOUT_FILENO, "%s\n", text) < 0) {
        perror("Writing output");
        dup2(saved_stdout, STDOUT_FILENO);
        close(saved_stdout);
        return 1;
    }

    if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
        perror("Restore stdout");
        close(saved_stdout);
        return 1;
    }

    close(saved_stdout);

    printf("Text successfully appended to %s\n", filename);

    return 0;
}
```

Topic 4 : To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```
vennela_07@VennelaPC:~$ mkdir stderr_redirection
vennela_07@VennelaPC:~$ cd stderr_redirection
vennela_07@VennelaPC:~/stderr_redirection$ nano stderr_redirection.c
vennela_07@VennelaPC:~/stderr_redirection$ gcc stderr_redirection.c -o stderr_redirection
vennela_07@VennelaPC:~/stderr_redirection$ ./stderr_redirection
Enter command followed by 2> filename: gcc test.c 2> errors.txt
Error output saved to errors.txt
vennela_07@VennelaPC:~/stderr_redirection$ cat errors.txt
Simulated error: command 'gcc test.c' failed.
vennela_07@VennelaPC:~/stderr_redirection$ ./stderr_redirection
Enter command followed by 2> filename: gcc test.c
Error: stderr redirection operator 2> not found.
vennela_07@VennelaPC:~/stderr_redirection$ gcc test.c 2>
-bash: syntax error near unexpected token `newline'
vennela_07@VennelaPC:~/stderr_redirection$ gcc test.c 2> /root/errors.txt
-bash: /root/errors.txt: Permission denied
```

GNU nano 7.2 stderr_redirection.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(void) {
    char input[256];
    char filename[100];
    char *symbol;
    int fd, saved_stderr;

    printf("Enter command followed by 2> filename: ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    symbol = strstr(input, "2>");

    if (symbol == NULL) {
        fprintf(stderr, "Error: stderr redirection operator 2> not found.\n");
        return 1;
    }

    *symbol = '\0';

    char *command = input;
    while (*command == ' ' || *command == '\t')
        command++;

    char *end = command + strlen(command);
    while (end > command && (end[-1] == ' ' || end[-1] == '\t'))
```

```c
    char *file = symbol + 2;
    while (*file == ' ' || *file == '\t')
        file++;

    if (*command == '\0') {
        fprintf(stderr, "Error: Missing command.\n");
        return 1;
    }

    if (*file == '\0') {
        fprintf(stderr, "Error: Missing filename.\n");
        return 1;
    }

    if (strchr(file, ' ') != NULL || strchr(file, '\t') != NULL) {
        fprintf(stderr, "Error: Use a filename without spaces.\n");
        return 1;
    }

    snprintf(filename, sizeof(filename), "%s", file);

    saved_stderr = dup(STDERR_FILENO);
    if (saved_stderr == -1) {
        perror("dup");
        return 1;
    }
```

```c
    }

    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("Error opening error-output file");
        close(saved_stderr);
        return 1;
    }

    if (dup2(fd, STDERR_FILENO) == -1) {
        perror("dup2");
        close(fd);
        close(saved_stderr);
        return 1;
    }

    close(fd);

    /* Demonstrate an error written to stderr. */
    fprintf(stderr, "Simulated error: command '%s' failed.\n", command);
    fflush(stderr);

    if (dup2(saved_stderr, STDERR_FILENO) == -1) {
        close(saved_stderr);
        return 1;
    }

    close(saved_stderr);

    printf("Error output saved to %s\n", filename);
    return 0;
}
```